

Making the RowHammer Simulator More Realistic

15 proposed improvements, each grounded in published measurements

gem5 DRAM RowHammer simulator · next-phase roadmap

Every claim on the following slides points to a specific figure, table or section of a named paper.

Where the simulator stands today

Four features are built, verified and pushed. Everything that follows extends this base rather than replacing it.

01

Weak-cell buckets

Rows split 30% weak and 70% non-weak, then cells binned into four vulnerability categories (X, Y, Z, P), each with its own flip probability.

02

Temperature and canary cells

A baseline weak set W0, plus 2% canary cells that turn weak only inside one specific temperature band, following SpyHammer.

03

Blast radius

Victims beyond the immediate neighbour, with flip probability scaled by a decaying distance factor, following BlockHammer.

04

Hardware aging

Extra cells wear into the weak set over time, with $|W(t)|$ growing linearly with the age of the part in weeks.

What is still missing is the physics that decides **WHEN** a flip happens, **WHICH** cell flips and in which direction, and **WHAT** a real system would actually observe once error correction has had its say.

The 15 improvements at a glance

A When a flip is triggered

- 1 Minimum hammer count before anything flips
- 2 Refresh rate changes the error count
- 3 RowPress: holding a row open is its own attack
- 4 Irregular hammer patterns, not a steady beat

B Which cell flips, and which way

- 5 Flips have a fixed physical direction
- 6 Stored data changes the flip count
- 7 Leaky cells are not hammer-vulnerable cells
- 8 Flips cluster instead of sprinkling evenly

C How far the damage reaches

- 9 Half-Double: reaching a victim through its neighbour
- 10 Reach grows with each DRAM generation

D Failures that develop over time

- 11 Leakage failures, including variable retention time
- 12 Calibrate aging against measured data

E Defenses and honest reporting

- 13 Model the defense that ships in real DRAM
- 14 Pluggable, comparable mitigations
- 15 Report flips the way ECC would see them

Each of the next 15 slides covers one improvement: what the research found, where our simulator falls short, what we would change, and why it matters — with the supporting figure taken straight from the paper.

01 Require a minimum number of hammers

Group A · When a flip is triggered

WHAT THE RESEARCH FOUND

Kim's team hammered rows on 129 real DDR3 modules and found something our simulator doesn't reproduce: below a certain number of activations, absolutely nothing happens. They measured that floor at 139,000 activations on one module, 155,000 on another and 284,000 on a third. Stay under it and the row is untouched; cross it and errors climb steeply. Figure 6 shows that cliff.

WHERE OUR SIMULATOR FALLS SHORT

We roll the dice on every hammer, so even a light workload can produce flips. The model has no concept of "not hammered enough yet."

WHAT WE WOULD CHANGE

Add an activation-count threshold to DRAMInterface.py, tracked per row alongside the hammer count we already keep. Below the threshold, return a flip probability of zero outright; above it, scale the existing category probability by how far past the threshold we are. Because that floor has dropped more than 10x since 2014 as cells shrank, make it a per-generation setting rather than one hard-coded number.

FROM THE PAPER

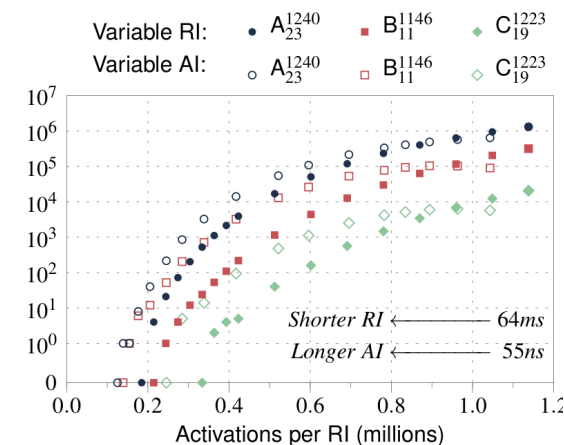


Figure 6. Number of errors vs. number of activations

Kim et al., "Flipping Bits in Memory Without Accessing Them," ISCA 2014 — Figure 6 and §6.2 "Number of Activations," p. 8. Scaling over generations: Mutlu & Kim, "RowHammer: A Retrospective," IEEE TCAD 2019, §2.

WHY IT MATTERS

Without a floor, every number we report would end up being an overestimate.

02 Make the refresh rate matter

Group A · When a flip is triggered

WHAT THE RESEARCH FOUND

DRAM rewrites every row on a timer. Kim varied that timer from 128 ms down to 16 ms while hammering, and errors fell by orders of magnitude as refresh got more frequent, and below a threshold interval they vanished entirely. Figure 4 is that measurement, and it is the reason “just refresh more often” is the oldest RowHammer defense there is.

WHERE OUR SIMULATOR FALLS SHORT

Refresh timing has no effect on our flip probability whatsoever, so we cannot answer “would refreshing twice as often have stopped this attack?”

WHAT WE WOULD CHANGE

Make flip probability a function of the refresh window: scale it by the ratio of the configured refresh interval to the 64 ms baseline Kim tested against. gem5 already models refresh timing inside the DRAM interface, so this is a matter of wiring a signal we already have into the flip decision — not inventing a new one.

FROM THE PAPER

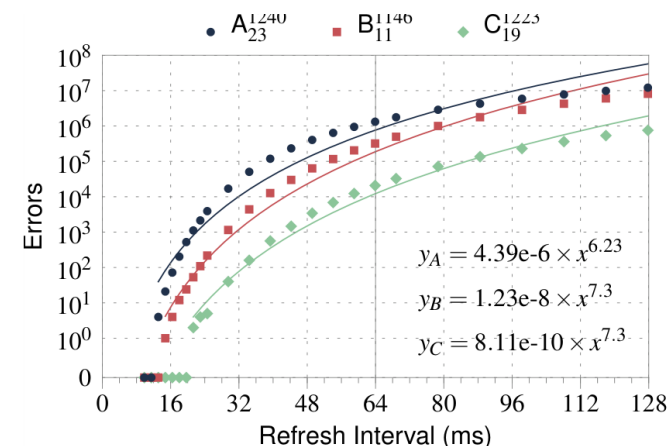


Figure 4. Number of errors as the refresh interval is varied

Kim et al., ISCA 2014 — Figure 4 and §6.2 “Refresh Interval,” p. 7. Errors drop by orders of magnitude between a 128 ms and a 16 ms refresh interval.

WHY IT MATTERS

It turns refresh interval into a knob we can sweep, which is one of the first questions one would ask about a proposed defense.

03

RowPress: holding a row open is its own attack

Group A · When a flip is triggered

WHAT THE RESEARCH FOUND

This is the newest and, to my reading, most important result on the list. Luo's group showed that simply keeping the aggressor row open for longer disturbs its neighboring rows enough to cause bitflips, using one to two orders of magnitude fewer activations than classic hammering, and in the extreme case a single activation is enough. It is a separate physical mechanism, not hammering done faster.

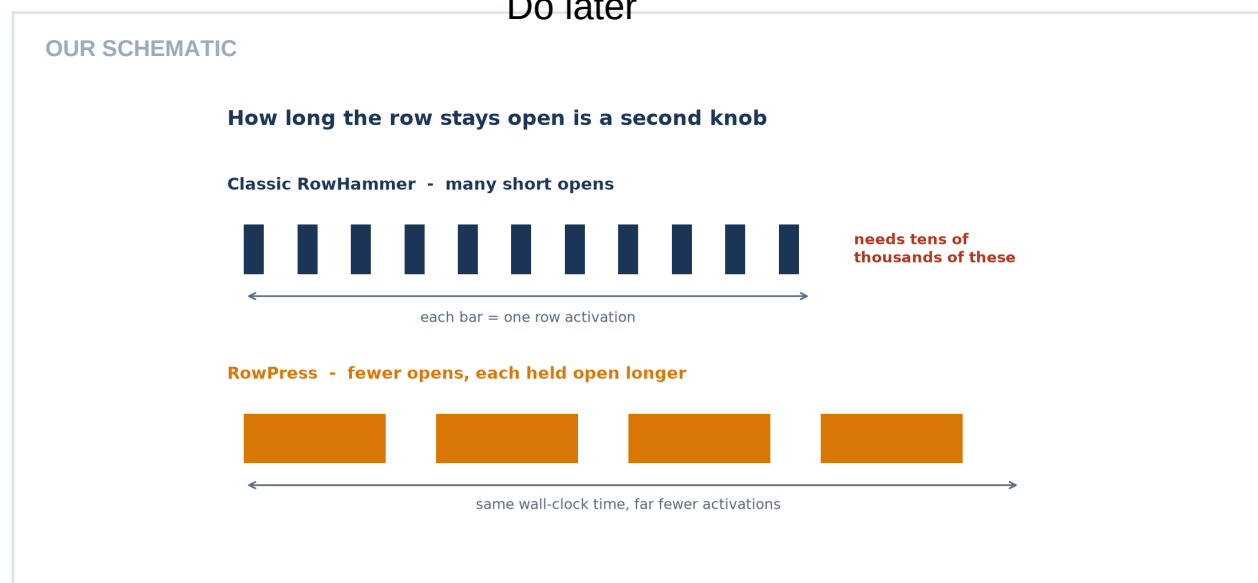
WHERE OUR SIMULATOR FALLS SHORT

We count how many times a row is hammered and nothing else. How long it stays open is invisible to the model, so an entire read-disturbance mechanism is missing.

WHAT WE WOULD CHANGE

Add row-open time as a second input to `getCellFlipProbability`, sitting alongside activation count. A longer open raises the per-activation flip chance, so a handful of long opens can end up as dangerous as a flood of short ones.

Do later



Luo et al., "RowPress: Amplifying Read Disturbance in Modern DRAM Chips," ISCA 2023 — §1 and §4 (minimum activation count vs. aggressor-row-on time). The diagram above is my own schematic, not theirs.

WHY IT MATTERS

Any attack the simulator currently judges "too slow to matter" might be trivially achievable this way, and we would never see it.

04 Hammer in irregular patterns, not a steady beat

Group A · When a flip is triggered

WHAT THE RESEARCH FOUND

Kim showed the ordering of accesses decides whether errors appear at all: open-read-close and open-write-close cause disturbance, while other orderings cause none (Table 4). Modern attacks push this much further. Blacksmith deliberately varies the phase, frequency and amplitude of hammering, and that irregularity produced bit flips on 100% of the DDR4 modules they tested.

WHERE OUR SIMULATOR FALLS SHORT

Our TrafficGen configs hammer at a constant rhythm, which is the easiest possible pattern for any tracker to notice and shut down.

WHAT WE WOULD CHANGE

Extend the traffic-generator configs to take a non-uniform aggressor schedule per-aggressor phase, frequency and amplitude values instead of one fixed interval. The hammering loop itself is unchanged; only the timing source becomes configurable.

FROM THE PAPER

Access Pattern	Disturbance Errors?
1. $(open-read-close)^N$	Yes
2. $(open-write-close)^N$	Yes
3. $open-read^N-close$	No
4. $open-write^N-close$	No

Table 4. Access patterns that induce disturbance errors

Kim et al., ISCA 2014 — Table 4 and §6.1 “Access Pattern,” p. 7. Frequency-domain patterns: Jattke et al., “BLACKSMITH: Scalable Rowhammering in the Frequency Domain,” IEEE S&P 2022, §V.

WHY IT MATTERS

Defenses are tuned to catch steady hammering. If we only ever generate the steady kind, any mitigation we add later will look far stronger than it actually is.

05 A flip has a direction, and it is always the same one

Group B · Which cell flips, and which way

WHAT THE RESEARCH FOUND

Hammering only ever drains charge. It pushes a cell from full toward empty, never the reverse. But cells are not all wired the same way. Liu's Figure 5 shows true-cells and anti-cells (where a full state is 0), which store a logical 1 using opposite physical states. So one physical drain shows up as $1 \rightarrow 0$ in some cells and $0 \rightarrow 1$ in others, and Kim confirmed only charged cells ever fail.

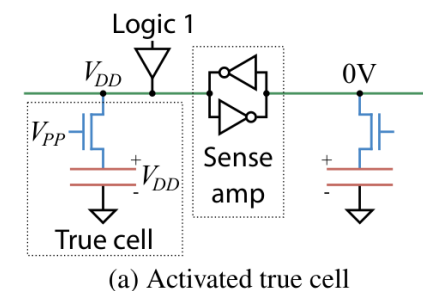
WHERE OUR SIMULATOR FALLS SHORT

A flip in our model has no direction. We record that a bit changed, not whether it could physically change that way given what was stored there.

WHAT WE WOULD CHANGE

Give every cell an orientation bit, derived from the same deterministic splitmix64 hash we already use for the temperature and aging models, so it costs no extra stored state. Then only permit a flip when that cell is currently in the drainable state, which means knowing the stored value, so this naturally lands together with improvement 6.

FROM THE PAPER



Do now

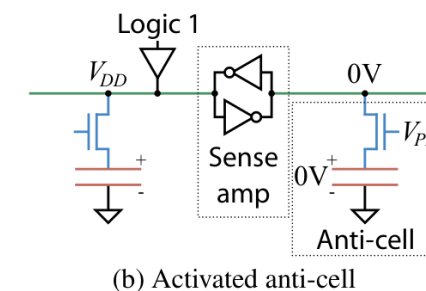


Figure 5: True and anti-cells

Liu et al., "An Experimental Study of Data Retention Behavior in Modern DRAM Devices," ISCA 2013 — Figure 5 "True and anti-cells," p. 5. Charge-drain-only physics: Kim et al., ISCA 2014, §6.4, p. 10.

WHY IT MATTERS

Roughly half the flips in our current output are physically impossible. Direction also decides exploitability: an attacker usually needs a specific bit to move a specific way.

06

The stored data changes how many flips you get

Group B · Which cell flips, and which way

Do now

WHAT THE RESEARCH FOUND

Identical hammering gives wildly different error counts depending on what is written in the rows. Kim's Table 6 is stark: on module A23, the RowStripe pattern produced 1,318,603 errors where Solid produced 112,123 — roughly a 12x swing from the data alone. Liu found the same kind of sensitivity on the retention side and named it data pattern dependence.

WHERE OUR SIMULATOR FALLS SHORT

Our flips are entirely data-agnostic, so every workload behaves like the average case and a worst-case pattern cannot be expressed at all.

WHAT WE WOULD CHANGE

Make flip probability depend on the aggressor and victim data patterns: a multiplier inside `getCellFlipProbability`, keyed on whether the neighbouring rows hold the same or the opposite value from the victim cell, calibrated so RowStripe-like patterns land near the ~12x that Kim measured.

FROM THE PAPER

Module	TESTBULK(DP) + TESTBULK(~DP)			
	Solid	RowStripe	ColStripe	Checkered
A ₂₃	112,123	1,318,603	763,763	934,536
B ₁₁	12,050	320,095	9,610	302,306
C ₁₉	57	20,770	130	29,283

Table 6. Number of errors for different data patterns

Kim et al., ISCA 2014 — Table 6 and §6.4 “Data Pattern Dependence,” p. 10. Retention-side equivalent: Liu et al., ISCA 2013, §5. These are standard test data patterns: the specific 1s and 0s you deliberately write into memory before hammering, to see how the pattern itself affects error rates

WHY IT MATTERS

Attackers choose their data pattern. Modelling only the average systematically understates the worst case — which is exactly the case a security study exists to examine.

07 Stop conflating leaky cells with hammer-vulnerable cells

Group B · Which cell flips, and which way

WHAT THE RESEARCH FOUND

It seems obvious that the ‘leakiest’ cells would also be the easiest to hammer, so Kim tested it directly. Out of roughly a million weak (leaky) cells per module, only 700, 220 and 19 were also RowHammer victims. They concluded the two are essentially independent: the coupling that causes hammering is a different physical path from the leakage that causes retention failure.

A leaky cell is weak on its own. It slowly loses its stored charge over time just from sitting there, with no hammering involved. This is a retention problem, like a bucket with a tiny hole in it.

A victim cell is a cell that flips specifically because a neighboring row got hammered: a disturbance problem caused by an external attack, not a leaky bucket.

WHERE OUR SIMULATOR FALLS SHORT

Our model draws both from a single weak pool, so a leaky cell is automatically hammer-prone. That is a modelling error, not a missing feature.

WHAT WE WOULD CHANGE

Split the weak-cell state into two independent sets (a retention set and a hammer-victim set) each seeded from its own salt derived from the root seed, exactly the pattern we already use for tempSeedSalt and agingSalt. A cell can then be in one set, both, or neither, which is what the measurement says.

Assume they are the same, change

later

FROM THE PAPER

Victim Cells $\neq$ Weak Cells. Although the retention time of every DRAM cell is required to be greater than the $64ms$ minimum, different cells have different retention times. In this context, the cells with the shortest retention times are referred to as *weak cells* [45]. Intuitively, it would appear that the weak cells are especially vulnerable to disturbance errors since they are already leakier than others. On the contrary, we did not find any strong correlation between weak cells and victim cells. We searched for a module’s weak cells by neither accessing nor refreshing a module for a generous amount of time (10 *seconds*) after having populated it with either all ‘0’s or all ‘1’s. If a cell was corrupted during this procedure,

Kim et al., ISCA 2014 — §7 “Victim Cells $\neq$ Weak Cells,” p. 10. The paragraph above is reproduced directly from that section.

WHY IT MATTERS

A correctness fix, and the cheapest item here because the two-salt infrastructure exists already. Everything layered on the weak-cell model inherits this error until it is fixed.

08

Flips cluster; they do not sprinkle evenly

Do now.

Group B · Which cell flips, and which way

WHAT THE RESEARCH FOUND

Vulnerable cells bunch tightly into specific regions of a row while neighbouring areas stay completely immune. Kim's Figure 7 shows a single aggressor row disturbing as many as 110 cells at once, and Table 5 counts 181,856 words holding 2 flips, 2,248 holding 3, and 18 holding 4 on one module.

WHERE OUR SIMULATOR FALLS SHORT

We place flips independently across the row, so two of them landing inside the same 64-bit word essentially never happens.

WHAT WE WOULD CHANGE

Add spatial correlation to the weak-cell assignment: hash on a coarse region index as well as the individual cell index, so vulnerability arrives in bursts rather than evenly. Tune the burst size against Kim's measured distribution of words carrying 1, 2, 3 and 4 flips.

FROM THE PAPER

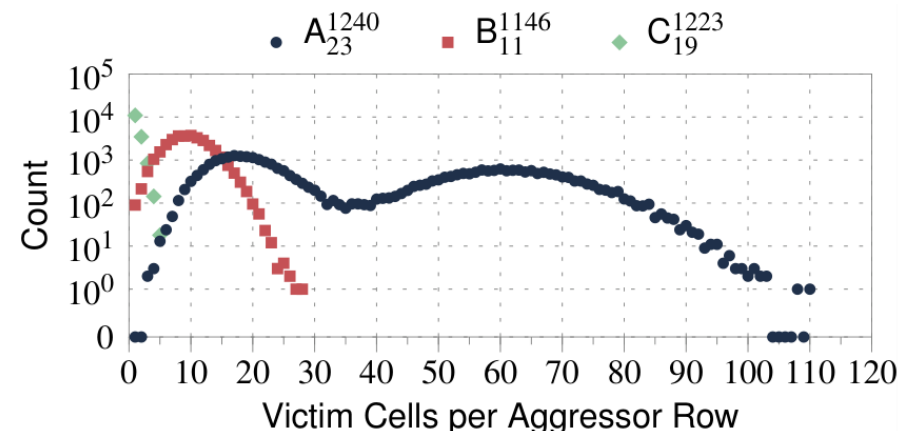


Figure 7. How many cells are affected by an aggressor row?

Kim et al., ISCA 2014 — Figure 7, p. 8 (up to 110 victim cells per aggressor row) and Table 5, p. 8 (181,856 / 2,248 / 18 words with 2 / 3 / 4 flips on module A23).

WHY IT MATTERS

This is a prerequisite for improvement 15. If flips never collide inside a word, ECC (Error Correcting Code) corrects every one and we would conclude ECC is a perfect defense, which Kim explicitly disproves.

09

Half-Double: reaching a victim through its neighbour

Do later with gem-5

Group C · How far the damage reaches

WHAT THE RESEARCH FOUND

Picture three rows in a line: Aggressor → Middle → Victim (rows at distance 1 and distance 2 from the target).

The classic finding is: hammering the row right next to a victim (distance 1) causes flips, and hammering a row two rows away (distance 2) causes far fewer, weaker flips distance just fades the effect out, like a stone dropped in water making smaller ripples farther away.

Half-Double found something surprising: if you hammer the distance-2 row heavily, and also sneak in just a handful of extra accesses to the middle row in between, the damage to the distant victim jumps way up: much more than you'd expect from either action alone. Thus, the middle row acts like a relay or an amplifier, not just a passive gap.

WHERE OUR SIMULATOR FALLS SHORT

Our blast-radius model applies a fixed fade-with-distance factor. Distance-1 and distance-2 activity never interact; each just contributes on its own.

WHAT WE WOULD CHANGE

Extend `blastRadiusFactor` and `checkBlastRadiusVictims` with a coupling term, so a small number of distance-1 accesses amplifies distance-2 hammering instead of merely adding to it. The distance bookkeeping we need is finished, so this is mostly a change to how the factor is computed.

FROM THE PAPER

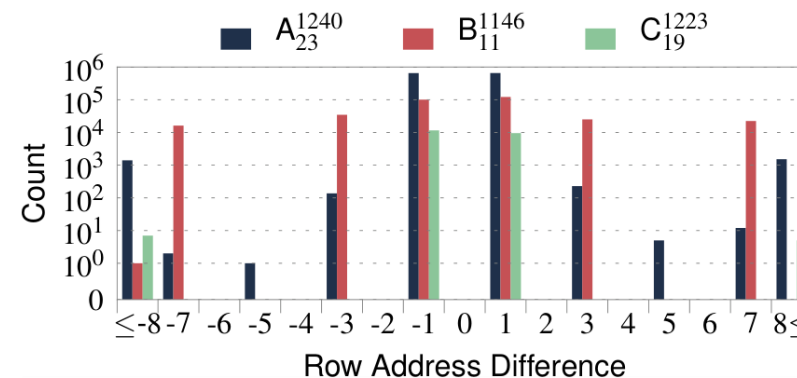


Figure 9. Which rows are affected by an aggressor row?

Baseline distance histogram: Kim et al., ISCA 2014 — Figure 9, p. 8. Distance-2 coupling: Kogler et al., "Half-Double: Hammering From the Next Row Over," USENIX Security 2022, §3.

Note: One access = one read or write to an address in that row (which forces an open→use→close cycle, per what we covered in #4).

"Hammering" a row just means hitting it with a huge number of these accesses back-to-back, as fast as possible. e.g., 139,000+ activations, per improvement 1.

WHY IT MATTERS

The cheapest high-value item after 7. It also matters because Half-Double specifically defeats defenses that only watch a row's immediate neighbours.

10

Reach grows with each DRAM generation

Group C · How far the damage reaches

WHAT THE RESEARCH FOUND

In the DDR3 parts Kim tested, victims sat almost entirely within two rows of the aggressor only 2.53%, 0.0122% and 0.00649% of aggressor rows reached three rows or further (Figure 8). But Orosa's characterisation of newer low-power nodes found flips as far as six rows away. The coupling gets longer-ranged as cells shrink.

WHERE OUR SIMULATOR FALLS SHORT

Blast radius is a fixed number the user types into the config, with no connection to which DRAM generation is being simulated.

WHAT WE WOULD CHANGE

Tie the default blast radius and its distance factors to the DRAM generation already selected in the gem5 config so choosing a newer part automatically widens the damage envelope instead of relying on the user to know the right value.

FROM THE PAPER

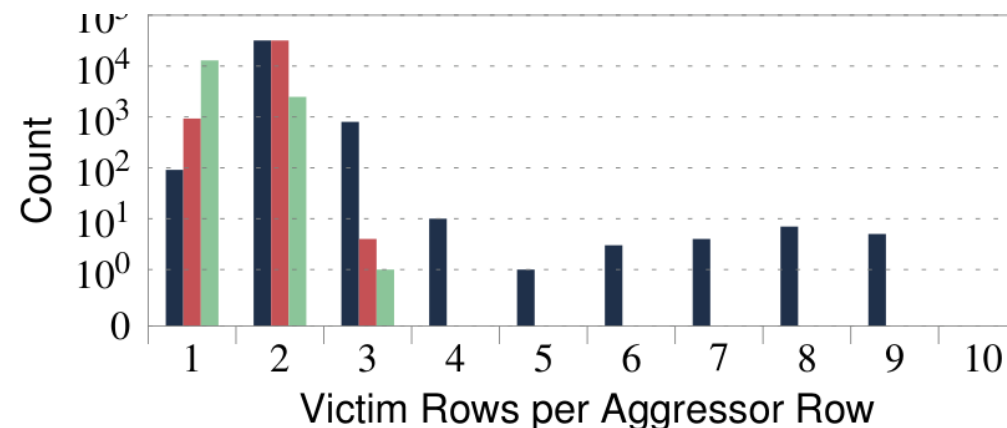


Figure 8. How many rows are affected by an aggressor row?

Kim et al., ISCA 2014 — Figure 8, p. 8. Wider reach at newer technology nodes: Orosa et al., "A Deeper Look into RowHammer's Sensitivities," MICRO 2021, §4.

WHY IT MATTERS

Getting a realistic result currently requires already knowing the answer. Tying reach to the part removes a whole class of quiet misconfiguration.

11

Add leakage failures, including the ones that come and go

Group D · Failures that develop over time

WHAT THE RESEARCH FOUND

Quite apart from hammering, cells leak and lose data. Worse, Liu found many cells randomly switch between reliable and leaky states over hours variable retention time. Figure 11 tracks one cell's retention time jumping around across 10 hours. Because of it, testing a chip once identifies under 15% of its bad cells. Leakage also roughly doubles for every 10 °C.

WHERE OUR SIMULATOR FALLS SHORT

We model no retention failures at all, and our temperature knob currently affects only hammer weak cells.

WHAT WE WOULD CHANGE

Add a retention failure path: a per-cell retention time plus stochastic switching between retention states as simulated time advances, wired into the temperature model we already built so both mechanisms share one knob rather than each having their own.

FROM THE PAPER

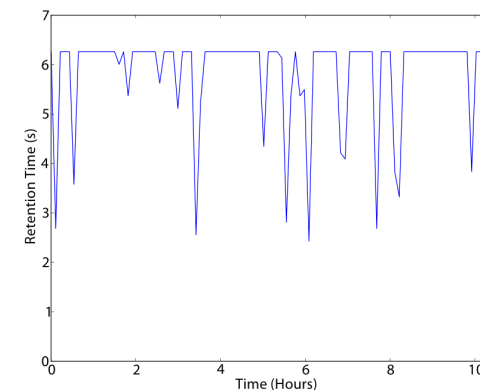


Figure 11: Retention time of a typical VRT cell

Liu et al., ISCA 2013 — Figure 11 “Retention time of a typical VRT cell,” p. 9 (§6); profiling coverage below 15% in §1; temperature effect in Figure 7b, p. 7.

WHY IT MATTERS

It is a second, independent failure mechanism real systems must handle, and it is what makes refresh and memory-scrubbing defenses testable. It also gives improvement 7's retention set a job.

WHAT THE RESEARCH FOUND

Bepary's group artificially aged real DRAM and measured retention errors at three points 0, 8 and 16 hours of accelerated aging across several vendors. Errors climbed consistently in every bank as the part wore (Figure 6), tracking the wear-out arm of the classic bathtub curve (Figure 2). Roughly 1% of cells sit in a high-leakage tail and dominate the effect.

WHERE OUR SIMULATOR FALLS SHORT

Our aging rate of $1e-5$ per week is a placeholder I picked so the trend would be visible on a graph. It is not fitted to anything.

WHAT WE WOULD CHANGE

Fit agingRate to Bepary's measured curve by matching our simulated increase against their 0 / 8 / 16-hour points, and add the ~1% high-leakage tail as a separate population that wears out faster than the rest of the bank rather than folding it into one uniform rate.

FROM THE PAPER

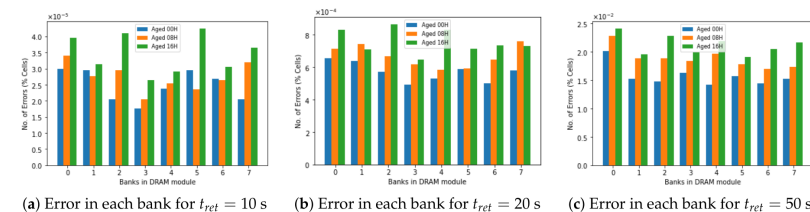


Figure 6. Retention error for 0x00 pattern in each bank with aging in a Hynix 2GB DRAM module.

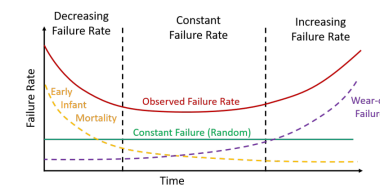


Figure 2. A classical bathtub curve represents common device failure characteristics [1].

Bepary et al., "DRAM Retention Behavior with Accelerated Aging in Commercial Chips," Applied Sciences 12(9):4332, 2022 — Figure 6 and §4.3.2, p. 11; bathtub curve in Figure 2, p. 4.

WHY IT MATTERS

The aging feature currently produces a plausible-looking trend with no claim to being correct. Fitting it to published measurements is what makes the result citable.

WHAT THE RESEARCH FOUND

Every modern DDR4 and DDR5 chip already contains a tracker that watches for hammered rows and refreshes their neighbours early. Frigo's team showed how it is beaten: hammer many rows at once (up to 19) and the tracker's limited slots overflow, so most aggressors are never refreshed. Phoenix did the same to DDR5 in 2025, getting root in 109 s on 15 of 15 modules.

WHERE OUR SIMULATOR FALLS SHORT

We model no in-DRAM defense at all, so every attack in our simulator succeeds against a chip that in practice would have resisted it.

WHAT WE WOULD CHANGE

Add a tracker model with a configurable slot count and sampling rule, plus multi-aggressor attack configs so the number of simultaneously hammered rows becomes a sweep parameter. Then report, per run, whether the pattern actually got through the tracker or was caught.

OUR SCHEMATIC

Why many-sided hammering defeats the built-in defense

Few aggressors: tracker keeps up



3 rows hammered

tracker (4 slots):



every hammered row is caught and its neighbours get refreshed early - attack fails

Many aggressors: tracker overflows



12 rows hammered

tracker (4 slots):



only 4 of 12 fit - the rest are never refreshed, and bits flip - attack succeeds

Frigo et al., "TRRespass: Exploiting the Many Sides of Target Row Refresh," IEEE S&P 2020, §V (up to 19 aggressors). DDR5: "Phoenix," 2025. The diagram above is my own schematic, not theirs.

WHY IT MATTERS

Without it our success rates describe hardware nobody ships. With it we get the natural headline result: how many aggressors does it take to break the defense?

14

Make defenses pluggable so they can be compared

Do now. May need gem 5. Make different files for different defences

Group E · Defenses and honest reporting

WHAT THE RESEARCH FOUND

The literature has settled into a standard menu: increased refresh rate, physical isolation, probabilistic reactive refresh such as PARA, counter-based tracking such as Graphene, and proactive throttling. BlockHammer's Table 6 lines them up across coverage, compatibility with commodity chips, scaling and determinism — and shows no single scheme wins on all four.

WHERE OUR SIMULATOR FALLS SHORT

The simulator reports how many bits flipped. It has no notion of a defense being present, so it cannot compare any two of them.

WHAT WE WOULD CHANGE

Implement mitigations as switchable modules behind one interface, starting with and one counter-based scheme, then report flips prevented and the overhead paid for each. The config picks the scheme the same way it currently picks a temperature.

FROM THE PAPER

Approach	Mechanism	Comprehensive Protection	Compatible w/ Commodity DRAM Chips	Scaling with RowHammer Vulnerability	Deterministic Protection
Increased Refresh Rate	[2, 73]	✓	✓	✗	✓
Physical Isolation	CATT [14]	✗	✗	✗	✓
	GuardION [148]	✗	✗	✗	✓
	ZebRAM [78]	✗	✗	✗	✓
Reactive Refresh	ANVIL [5]	✗	✗	✗	✓
	PARA [73]	✓	✗	✗	✗
	PRoHIT [137]	✓	✗	✗	✗
	MRLoc [161]	✓	✗	✗	✗
	CBT [132]	✓	✗	✗	✓
	TWiCe [84]	✓	✗	✗	✓
Proactive Throttling	Graphene [113]	✓	✗	✓	✓
	Naive Thrott. [102]	✓	✓	✗	✓
	Thrott. Supp. [40]	✓	✗	✗	✓
	BlockHammer	✓	✓	✓	✓

Table 6: Comparison of RowHammer mitigation mechanisms.

Yan et al., “BlockHammer: Preventing RowHammer at Low Cost by Blacklisting Rapidly-Accessed DRAM Rows,” HPCA 2021 — Table 6 and §9, p. 11. PARA: Kim et al., ISCA 2014, §8.2 and Table 7, p. 11.

WHY IT MATTERS

This is what turns the tool from a bit-flip generator into a platform for evaluating defenses.

15 Report flips the way a real system would see them

Group E · Defenses and honest reporting

WHAT THE RESEARCH FOUND

Server memory uses SECDED error correction: one bad bit inside a 64-bit word is silently repaired, two or more is not. Kim counted exactly how many words carry 2, 3 or 4 flips (Table 5) and concluded plainly that SECDED is not failsafe against disturbance errors. Later work went further and attacked ECC memory outright.

WHERE OUR SIMULATOR FALLS SHORT

We report raw flip counts, which overstates the danger — most of those flips would be silently corrected before software ever saw them.

WHAT WE WOULD CHANGE

Add a post-processing layer over the flip log that groups flips by 64-bit word and sorts each word into corrected, detected-but-uncorrectable, or silently wrong. The flip model itself does not change at all — this is purely a reporting layer, which makes it low-risk to add.

FROM THE PAPER

Module	Number of 64-bit words with X errors			
	$X = 1$	$X = 2$	$X = 3$	$X = 4$
A_{23}	9,709,721	181,856	2,248	18
B_{11}	2,632,280	13,638	47	0
C_{19}	141,821	42	0	0

Table 5. Uncorrectable multi-bit errors (in bold)

Kim et al., ISCA 2014 — Table 5 and §6.3, p. 8: “we conclude that SECDED is not failsafe against disturbance errors.” See also Cojocar et al. (ECCploit), IEEE S&P 2019.

WHY IT MATTERS

Raw flip counts are not what a real machine experiences. Paired with improvement 8, this shows what an attacker could genuinely exploit rather than what the array technically did.

Where I would start

Rough thoughts on ordering rather than a fixed plan — happy to be talked out of any of it.

First, if we only get to a few

7 · 3 · 1 · 9 · 5

Number 7 is technically a bug, so I would not let it sit. RowPress (3) is the newest big finding and the biggest hole we have. Number 1 finally gives the model a reason to say “not yet.” And 9 and 5 are relatively simple, because they reuse code that is already written and tested: the blast-radius logic and the per-cell hashing.

Next, once those land

8 · 15 · 13 · 6 · 2

8 and 15 really have to travel together: clustering is what makes the ECC layer mean anything, and without ECC our flip counts are not what a real machine would report. 13 stops every attack from succeeding automatically. Then 6 and 2 make the model react to the stored data and to refresh timing.

Bigger jobs, better saved for later

11 · 14 · 4 · 10 · 12

11 is a whole second failure mechanism, so it is a project in its own right. 14 only really pays off once there are several mechanisms worth defending against. And 4, 10 and 12 are calibration and configuration work that gets easier once the rest is in place.

If we only have room for five, I would pitch 3, 7, 9, 13 and 15 — one real bug fix, the newest discovery, a cheap win, the defense that real chips actually ship with, and honest reporting.

Sources

PAPER		USED FOR
Kim et al.	Flipping Bits in Memory Without Accessing Them: An Experimental Study of DRAM Disturbance Errors. ISCA 2014.	1, 2, 4, 6, 7, 8, 9, 10, 14, 15
Liu et al.	An Experimental Study of Data Retention Behavior in Modern DRAM Devices. ISCA 2013.	5, 11
Bepary et al.	DRAM Retention Behavior with Accelerated Aging in Commercial Chips. Applied Sciences 12(9):4332, 2022.	12
Mutlu & Kim	RowHammer: A Retrospective. IEEE TCAD 2019.	1 (scaling)
Yağmurlu et al.	BlockHammer: Preventing RowHammer at Low Cost by Blacklisting Rapidly-Accessed DRAM Rows. HPCA 2021.	14
Luo et al.	RowPress: Amplifying Read Disturbance in Modern DRAM Chips. ISCA 2023.	3
Kogler et al.	Half-Double: Hammering From the Next Row Over. USENIX Security 2022.	9
Jattke et al.	BLACKSMITH: Scalable Rowhammering in the Frequency Domain. IEEE S&P 2022.	4
Frigo et al.	TRRespass: Exploiting the Many Sides of Target Row Refresh. IEEE S&P 2020.	13
Orosa et al.	A Deeper Look into RowHammer's Sensitivities. MICRO 2021.	10
Cojocar et al.	Exploiting Correcting Codes: On the Effectiveness of ECC Memory Against RowHammer Attacks. IEEE S&P 2019.	15
		20